University of Southampton

COMP6237 Data Mining
Lecture 6: Document Filtering

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS

University of Southampton

Lecture slides available here:

http://comp6237.ecs.soton.ac.uk/zh.html

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Document Filtering – Problem

- `spam` = unwanted / promotional / phishing-style messages
- `ham` = normal messages

```
"The seminar starts at 2pm in Lecture Theatre A."),
"Please submit your assignment via Blackboard."),
"Office hours are moved to Thursday this week."),
```

S1

Spam or ham?

```
"Please find the attached report."),
"Let's discuss this in tomorrow's meeting."),
"I have updated the document as requested."),
```

S2

Spam or ham?

```
"Happy birthday! Hope you have a great day."),
"Are you free this weekend?"),
"Let's catch up soon."),
```

S3

Spam or ham?

```
, "Security alert: unusual login detected. Reset password immediately."),
, "Your bank account has been compromised. Verify now."),
, "Update your payment details to avoid suspension."),
```

S4

Spam or ham?

```
, "Invest in crypto now and double your money."),
, "Guaranteed profit opportunity – act today."),
, "Low interest loan approved instantly."),
```

S5

Spam or ham?

```
, "Buy one get three free today only."),
, "Flash sale – 95% off everything."),
, "Cheap flights available now.")
```

S6

Spam or ham?

Document Filtering – Problem

Input: a document $d \in \mathcal{D}$ (email/SMS).
Output: a class label $c \in \mathcal{C} = \{\text{spam}, \text{ham}\}$.

Given labeled training data $\mathcal{S} = \{(d_i, c_i)\}_{i=1}^{N}$,

we learn a classifier $f : \mathcal{D} \to \mathcal{C}$ (equivalently a model of $p(c \mid d)$)

and predict for an unseen document d via $\hat{c}(d) = \arg\max_{c \in \mathcal{C}} p(c \mid d)$.

In this lecture we focus on:

- **Feature engineering:** $x = \phi(d) \in \mathbb{R}^V$ (e.g., Bag-of-Words / TF–IDF)
- **Naïve Bayes**
- **Fisher's method**

Document Filtering – Naïve Bayes

Start with *bag of words*

- simple tokenisation: split on non-letters
- simple pre-processing: convert all to lower case
- count only presence or absence of term

Make a table counting the number of times a term has appeared for each category

	money	viagra	tea	you	dear	...	Total
Spam	15	8	0	19	15	...	30
Ham	2	0	15	20	12	...	70

Count up the total number of documents

What about TF-IDF feature?

Demo: Document Filtering Ham or Spam

Document Filtering – Naïve Bayes

Now work out the conditional probability, i.e. the probability of a feature given a category:

If n_c is the number of documents in a category, and n_{fc} is the number of documents with feature f in category c..

$$p(f|c) = \frac{n_{fc}}{n_c}$$

	money	viagra	tea	you	dear	...	Total
Spam	15	8	0	19	15	...	30
Ham	2	0	15	20	12	...	70

Using the data above:
calculate $p(\text{“}viagra\text{”}|Spam)$

Document Filtering – Naïve Bayes

Now work out the conditional probability, i.e. the probability of a feature given a category:

If n_c is the number of documents in a category, and n_{fc} is the number of documents with feature f in category c..

$$p(f|c) = \frac{n_{fc}}{n_c}$$

	money	viagra	tea	you	dear	...	Total
Spam	15	8	0	19	15	...	30
Ham	2	0	15	20	12	...	70

Using the data above:
calculate $p(\text{“}viagra\text{”}|Spam) = 8/30 \approx 0.27$
calculate $p(\text{“}tea\text{”}|Ham)$

Document Filtering – Naïve Bayes

Now work out the conditional probability, i.e. the probability of a feature given a category:

If n_c is the number of documents in a category, and n_{fc} is the number of documents with feature f in category c..

$$p(f|c) = \frac{n_{fc}}{n_c}$$

	money	viagra	tea	you	dear	...	Total
Spam	15	8	0	19	15	...	30
Ham	2	0	15	20	12	...	70

Using the data above:
calculate $p(\text{“}viagra\text{”}|Spam) = 8/30 \approx 0.27$
calculate $p(\text{“}tea\text{”}|Ham) = 15/70 \approx 0.21$.. but..
calculate $p(\text{“}tea\text{”}|Spam)$

Document Filtering – Naïve Bayes

Now work out the conditional probability, i.e. the probability of a feature given a category:

If n_c is the number of documents in a category, and n_{fc} is the number of documents with feature f in category c..

$$p(f|c) = \frac{n_{fc}}{n_c}$$

	money	viagra	tea	you	dear	...	Total
Spam	15	8	0	19	15	...	30
Ham	2	0	15	20	12	...	70

Using the data above:
calculate $p(\text{“}viagra\text{”}|Spam) = 8/30 \approx 0.27$
calculate $p(\text{“}tea\text{”}|Ham) = 15/70 \approx 0.21$.. but..
calculate $p(\text{“}tea\text{”}|Spam) = 0/30 = 0$!.. does this make sense?

Document Filtering – Naïve Bayes

Smoothing probability estimates

Should "tea" *never* be expected to appear in Spam documents?

Need a better way to estimate the conditional probability that accounts for infrequently seen features (sample size too small)

Document Filtering – Naïve Bayes

Smoothing probability estimates

Should "tea" *never* be expected to appear in Spam documents?

Need a better way to estimate the conditional probability that accounts for infrequently seen features (sample size too small)

We introduce an *assumed* probability

- where there is little evidence
- can be based on some evidence

Produce a weighted estimate for the conditional probability based on the assumed and the raw computed probability

Document Filtering – Naïve Bayes

Using Bayesian Statistics, assuming that random variables are drawn from probability distributions rather than point samples

In this case, Spam/Ham classification for a feature f is binomial with a *beta* distributed **prior**

$$p_w(f|c) = \frac{weight \times assumed + count \times p_{raw}(f|c)}{count + weight}$$

where *count* is the number of times feature f occurs across all categories, *weight* is the strength we will give that assumption

In practice, the values for ***weight*** and ***assumed*** are found through testing to optimize performance. Reasonable starting points are **1** and **.5** for *weight* and *assumed*, respectively

https://dl.acm.org/doi/fullHtml/10.5555/636750.636753

Document Filtering – Naïve Bayes

However the conditional probability of the whole document is required

To do this we assume that all features are independent of each other.

This is what makes it Naïve Bayes

In this case the *features* are words, and as certain words are very likely to appear together this assumption is false. However in practice, it doesn't matter.

It will still work even if incorrect!

Document Filtering – Naïve Bayes

The Naïve assumption allows the conditional probability to be expressed as the product of all the conditional feature probabilities

$$p(d|c) = \prod_{f \in d} p(f|c)$$

This is the **likelihood** of the document given a category.

Posterior $\propto$ Likelihood Prior

Naive Bayes rules

$$P(c|d) = \frac{P(d|c)\,P(c)}{p(d)}$$

- P(c|d) = conditional probability of c given d (Posterior)
- P(d|c) = conditional probability of d given c (Likelihood)
- P(c) = prior probability of hypothesis/class c (Prior)
- P(d) = prior probability of data d (Evidence)

Thomas Bayes (1702–1761)

Document Filtering – Naïve Bayes

The Naïve assumption allows the conditional probability to be expressed as the product of all the conditional feature probabilities

$$p(d|c) = \prod_{f \in d} p(f|c)$$

This is the **likelihood** of the document given a category.

Posterior $\propto$ Likelihood Prior

Naive Bayes rules

$$P(c|d) = \frac{P(d|c)\,P(c)}{p(d)}$$

The **prior** can be calculated:

- Empirically, using the total No. docs in c divided by the total number
- or assuming all classes to be equally probably

$p(d)$ is constant, so therefore irrelevant, as same for all categories

Thomas Bayes (1702–1761)

Credit: Jo Grundy

Document Filtering – Naïve Bayes

The Naïve assumption allows the conditional probability to be expressed as the product of all the conditional feature probabilities

$$p(d|c) = \prod_{f \in d} p(f|c)$$

This is the **likelihood** of the document given a category.

When implementing, the product of a lot of very small numbers could lead to *floating point underflow*, so we take the logs and sum instead.

$$\log(p(c|d)) \propto \log(p(c)) + \sum_{f \in d} \log(p(f|c))$$

Document Filtering – Naïve Bayes

$$\text{posterior} \propto \text{likelihood} \times \text{prior}$$

$$\therefore p(c|d) \propto p(d|c) \times p(c)$$

$p(c|d)$ can be calculated for all categories using this formula
The most likely category is thus assigned to the document, i.e. with the largest $p(c|d)$. This is **maximum a posteriori**, MAP, and assumes the categories are equal.

But in spam filtering, mistakes are not equally bad:

- **False positive (FP)**: a *good* email (ham) is marked as spam → you might miss an important email. **High cost**.
- **False negative (FN)**: a spam email is marked as ham → annoying but usually **lower cost**.

Document Filtering – Naïve Bayes

Precision(spam) and Recall(spam) perspective

Treat **spam as the positive class**.

- **Precision(spam)** $= \dfrac{TP}{TP + FP}$

  "Of the emails I labeled spam, how many are truly spam?"

  → **FP hurts precision directly.**

  Because FP is costly, we typically want **high precision**.

- **Recall(spam)** $= \dfrac{TP}{TP + FN}$

  "Of all true spam emails, how many did I catch?"

  → **FN hurts recall.**

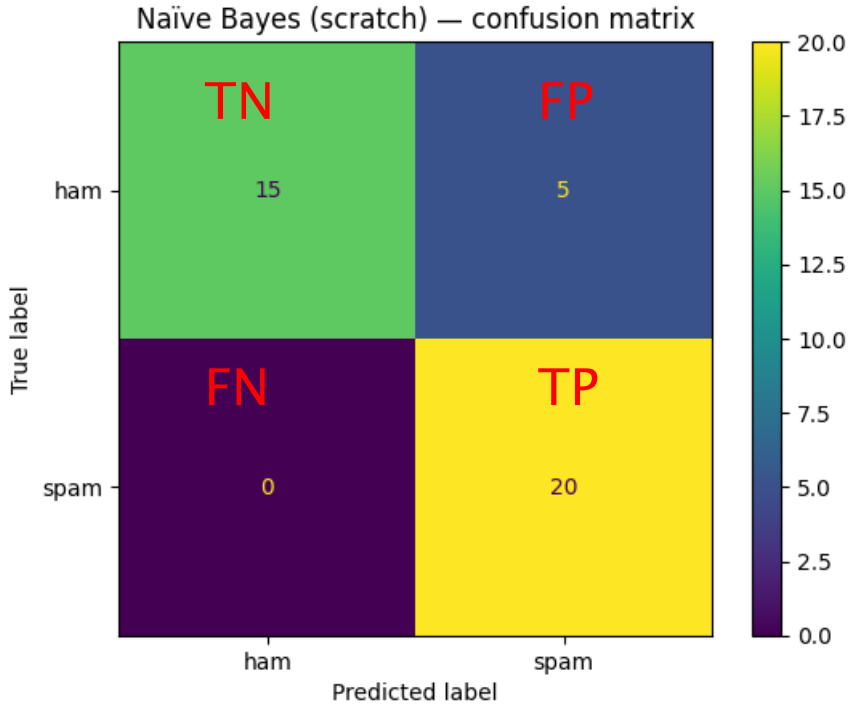
When you raise the threshold for predicting spam (to avoid FP):

- **FP decreases → precision(spam) increases**
- **FN increases → recall(spam) decreases**

So the slide's message is: in spam filtering, we often **prefer higher precision** (fewer legitimate emails wrongly blocked), even if that means **lower recall** (some spam slips through).

A common operational choice:

- "Only call it spam if very confident" → **high precision, lower recall**
- Or put uncertain ones into a "possible spam" folder (a different action with a different cost).




```
{'accuracy': 0.875,
 'precision(spam)': 0.8,
 'recall(spam)': 1.0,
 'f1(spam)': 0.8888888888888888}
```

Document Filtering – Naïve Bayes

In spam filtering, a **false positive** (ham classified as spam) is often more costly than a **false negative**.

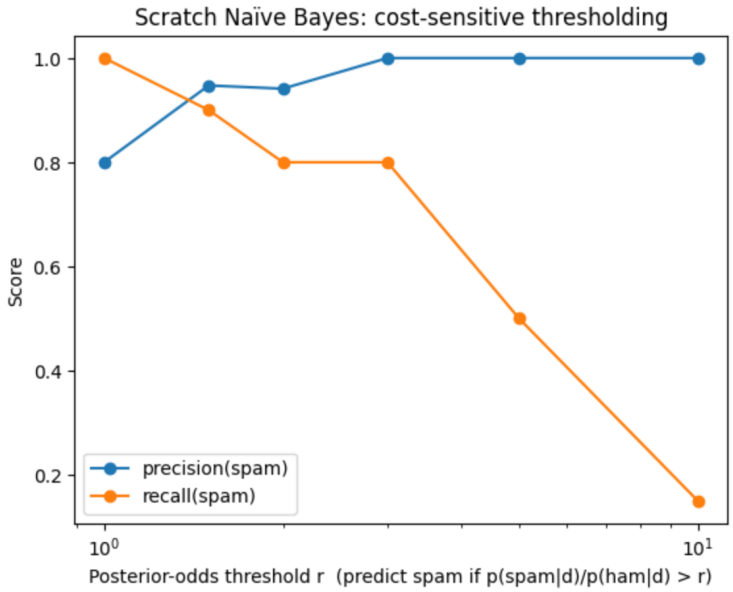
So instead of predicting spam whenever it has the larger posterior (MAP), we can require a higher posterior odds:

$$\frac{p(\text{spam}|d)}{p(\text{ham}|d)} > r \quad \Rightarrow \quad \text{predict spam.}$$

For two classes this is equivalent to:

$$p(\text{spam} \mid d) > \frac{r}{1+r}.$$

	r	equiv_p_spam	accuracy	precision(spam)	recall(spam)	f1(spam)
0	1.0	0.500000	0.875	0.800000	1.00	0.888889
1	1.5	0.600000	0.925	0.947368	0.90	0.923077
2	2.0	0.666667	0.875	0.941176	0.80	0.864865
3	3.0	0.750000	0.900	1.000000	0.80	0.888889
4	5.0	0.833333	0.750	1.000000	0.50	0.666667
5	10.0	0.909091	0.575	1.000000	0.15	0.260870



Demo: Document Filtering Ham or Spam

Document Filtering – Fisher's Method vs. Naïve Bayes

Naïve Bayes: uses feature likelihoods to compute whole document probability

$$\log(p(c|d)) \propto \log(p(c)) + \sum_{f \in d} \log(p(f|c))$$

Fisher's Method

- calculates probability of each category for each feature $p(c|f)$
- tests to see if combined probabilities are more or less likely than a random

This assumes independence of features

$$p(c|d) = K^{-1}(-2\sum_{i=1}^{k} \ln(p(c = C|f_i)), 2k) = K^{-1}(-2\ln(\prod_{i=1}^{k} p(c = C|f_i)), 2k)$$

where K^{-1} is the inverse chi-squared function

Document Filtering – Fisher's Method

To calculate category probabilities $p(c|f)$ for given feature:
Can use Bayes' Extended form, but need $P(c)$ for all c

$$p(c|f) = \frac{p(f|c_i)p(c_i)}{\sum_j p(f|c_j)p(c_j)}$$

$p(c_j)$ can be estimated from the data, or from an unbiased estimate, all $p(c_j)$ equally likely

Document Filtering – Fisher's Method

$$p(c|f) = \frac{p(f|c_i)p(c_i)}{\sum_j p(f|c_j)p(c_j)}$$

For a Spam/Ham classifier, there is no *a priori* reason to assume one or the other.

$$P(c) = P(Ham) = P(Spam) = 0.5$$

$$\therefore P(c|f) = \frac{P(f|c)}{(P(f|Ham) + P(f|Spam))}$$

Document Filtering – Fisher's Method

Fisher's method combines k probabilities ("p-values") $P(c = C|f_k)$ from each test in to a single test statistic, X^2

$$X^2_{2k} \sim -2 \sum \ln(p(c = C|f_i))$$

if the p-values are independent, this X^2 statistic will follow a *chi-squared* distribution in $2k$ degrees of freedom

So we can calculate a combined p-value

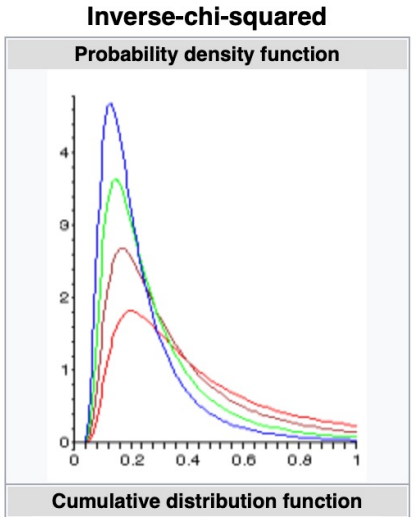
$$p_C = K^{-1}(-2 \sum_{i=1}^{k} \ln(p(c = C|f_i)), 2k) = K^{-1}(-2 \ln(\prod_{i=1}^{k} p(c = C|f_i)), 2k)$$

where K^{-1} is the inverse chi-squared function

PDF	$\frac{2^{-\nu/2}}{\Gamma(\nu/2)} x^{-\nu/2-1} e^{-1/(2x)}$

Parameters	$\nu > 0$

https://en.wikipedia.org



https://en.wikipedia.org

Document Filtering – Fisher's Method

Making classifications

$$I = \frac{1 + p_{Spam} - p_{Ham}}{2}$$

I tends to 1 if document is Spam, and 0 if it is Ham.

Demo: Document Filtering Ham or Spam

University of Southampton

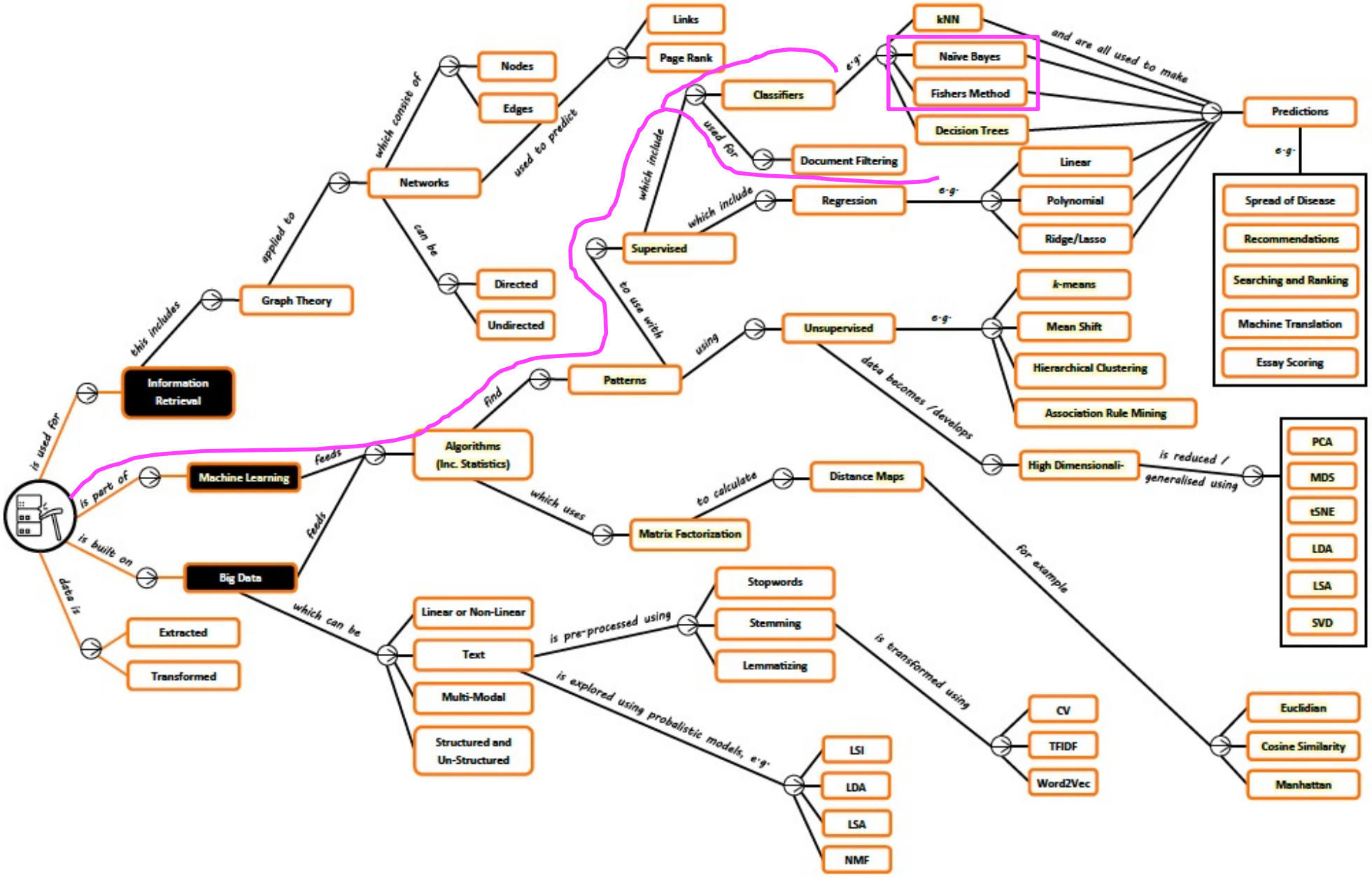
Document Filtering – Summary

- **Document filtering** is a supervised classification problem (e.g., spam vs ham).
- **Feature engineering** is critical:
  - **Bag of Words** is simple and effective
  - **TF–IDF** reweights words by distinctiveness
- **Naïve Bayes**
  - Estimate $p(f \mid c)$, assume conditional independence, and combine evidence by multiplication (equivalently, sum log-probabilities in practice).
  - Smoothing is essential when data is small.
- **Fisher's method**
  - Estimate $p(c \mid f)$ per feature and combine evidence via Fisher's χ^2 method.
  - Produces a natural spam score $I \in [0, 1]$.

> **Takeaway:** the choice of features is often as important as the choice of classifier.

University of Southampton

Document Filtering – **Roadmap**



Document Filtering – **Textbook**

University of Southampton

CHAPTER 6

Document Filtering

This chapter will demonstrate how to classify documents based on their contents, a very practical application of machine intelligence and one that is becoming more widespread. Perhaps the most useful and well-known application of document filtering is the elimination of spam. A big problem with the wide availability of email and the extremely low cost of sending email messages is that anyone whose address gets into the wrong hands is likely to receive unsolicited commercial email messages, making it difficult for them to read the messages that are actually of interest.

- Programming Collective Intelligence: Building Smart Web 2.0 Applications *T. Segaran*.

Document Filtering – **Learning Outcomes**

- **LO1**: Demonstrate an understanding of the fundamental concepts and approaches for document filtering, such as: (exam)
  - Understanding the basic ideas behind Naive Bayes and Fisher's methods
  - Applying Naïve Bayes and Fisher's Methods to classify documents as spam or not spam
  - Discussing the pros and cons of using Naive Bayes vs. Fisher's methods for document filtering
- **LO2**: Implement the learned algorithms for document filtering (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf*
- *ChatGPT or other AI-based techs*